DOT NET TECHNOLOGY

Pr Fatimazahra Barramou

Introduction

Module	Course	VH
Enterprise Application Development 2	C# Programming	20h
	.NET Technology	36h

Course Objectives

3

- Understand the fundamentals of C# language and the .NET runtime.
- Build ASP.NET Core MVC and Web API applications.
- Use Entity Framework Core to interact with relational databases.
- Apply object-oriented programming, clean architecture, and design patterns.
- Test and debug .NET applications using xUnit.
- Deploy applications using Docker and modern CI/CD workflows.
- Work in teams using Git and Agile principles.

Prerequisites

4

- Basic programming (Python, Java, or C#).
- Fundamentals of OOP.
- Basic SQL.
- Experience with Git is recommended.

Tools

5

- Visual Studio 2022 or VS Code.
- NET 8 SDK
- SQL Server / PostgreSQL
- Postman
- GitHub
- Docker Desktop

6

Chapter: Web Development with .NET

Presentation Outline

7

- Overview of modern web applications
- .NET ecosystem
- ASP.NET Core features
- MVC vs Web API vs Razor Pages

8

Overview of Modern Web Applications

What is a Web Application

9

A web application is a **software system accessible via a web browser,** designed to handle:

- User interaction
- Business logic
- Data persistence
- Communication over the HTTP protocol

Unlike **desktop applications,** web applications are:

- Platform-independent
- Accessible remotely
- Often distributed and scalable

Evolution of Web Applications

10

- Static Websites
  - HTML + CSS
  - No server-side logic
  - Content does not change dynamically
- Dynamic Web Applications
  - Server-side processing
  - Database-driven content
  - Examples: PHP, ASP.NET, JSP
- Modern Web Applications
  - Separation of concerns
  - Client–server architecture
  - APIs and microservices
  - Asynchronous communication (AJAX, REST)

Key Characteristics of Modern Web Applications

11

Modern web applications typically include:

- **Client–Server Architecture**
- **Stateless Communication** (HTTP)
- **Scalability**
- **Security** (authentication, authorization)
- **Maintainability**
- **Cross-platform compatibility**

Client–Server Architecture

12

A web application is based on **two main components:**

- **Client**
  - Web browser (Chrome, Edge, Firefox)
  - Sends HTTP requests
  - Renders HTML, CSS, JavaScript
- **Server**
  - Processes requests
  - Executes business logic
  - Accesses databases
  - Returns responses (HTML / JSON)

Client–Server Architecture

13

- **Request / Response Cycle**
  1. Client sends HTTP request
  2. Server processes request
  3. Server accesses data if needed
  4. Server sends HTTP response
  5. Browser displays the result
- **Important concepts**
  - Stateless communication
  - Each request is independent
  - Session & cookies used to maintain state

Multi-Tier Architecture

14

- Modern web applications are **layered** to improve:
  - Maintainability
  - Scalability
  - Security
  - Team collaboration

Multi-Tier Architecture

15

A typical web application is composed of:

- **Presentation Layer**
  - User Interface (UI)
  - HTML / CSS / JavaScript
  - Razor Views / SPA Frontend
- **Business Logic Layer**
  - Application rules
  - Validations
  - Services
- **Data Access Layer**
  - Database communication
  - ORM (Entity Framework Core)
  - SQL execution

Monolithic vs Modern Architectures

16

Monolithic vs Modern Architectures

- **Monolithic Application**
  - All components in one project
  - Single deployment
  - Simple but hard to scale
- **Advantages**
  - Easy to start
  - Simple deployment
- **Disadvantages**
  - Difficult maintenance
  - Poor scalability
  - Tight coupling

Modern Architectures - Overview

17

- **Layered Monolith**
  - MVC applications
  - Clear separation inside one application
- **API-Based Architecture**
  - Backend exposes REST APIs
  - Frontend consumes APIs
- **Microservices**
  - Independent services
  - Each service has its own database
  - Complex but scalable

MVC vs SPA

18

- **MVC Applications**
  - Server renders HTML
  - Logic handled on server
  - SEO friendly
  - Easier for beginners
- **SPA (Single Page Applications)**
  - Frontend frameworks (Angular, React)
  - Backend exposes APIs only
  - Rich interactivity
  - More complex

Why Backend Frameworks are Needed

19

- Without a framework:
  - Hard to manage routing
  - Security risks
  - Repetitive code
  - Poor maintainability
- Backend frameworks provide:
  - Routing
  - Security
  - Data access abstraction
  - Dependency injection
  - Logging & configuration

20

.NET Ecosystem

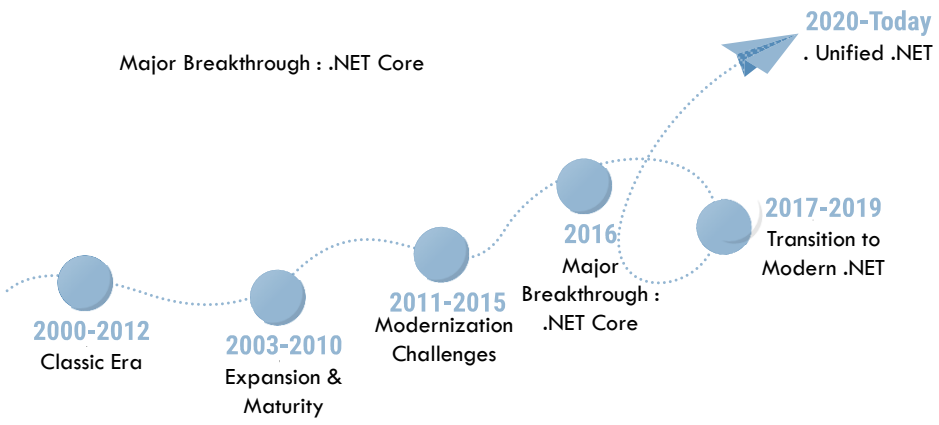
Introduction to the .NET Platform

21

- **.NET** is a **software development platform** created by **Microsoft** that allows developers to:
  - Build applications using multiple programming languages
  - Run applications on different operating systems
  - Share a common runtime and libraries
- **What .NET provides**
  - A runtime environment
  - A rich class library
  - Development tools
  - Cross-platform support

A Brief History of .Net

22



A Brief History of .Net

23

2000–2002: .NET Framework (Classic Era)

- Microsoft introduces .NET Framework
- Key components:
  - CLR (Common Language Runtime)
  - C# as a new modern language
  - ASP.NET for web development

A Brief History of .Net

24

2003–2010: Expansion & Maturity

- Introduction of major technologies:
  - **ASP.NET Web Forms**
  - **ADO.NET** (data access)
  - **WPF, WCF, WF** (desktop & services)
- **LINQ (2007)** revolutionizes data querying
- Strong focus on **Windows-only** development

A Brief History of .Net

25

2011–2015: Modernization Challenges

- Rise of:
  - Mobile apps
  - Cloud computing
  - Open-source ecosystems
- Limitations of classic .NET:
  - Heavy
  - Windows-dependent
- Microsoft prepares a major shift

A Brief History of .Net

26

2016: ASP.NET Core Revolution

- Release of **.NET Core**
- Key changes:
  - Cross-platform (Windows, Linux, macOS)
  - High performance
  - Open-source
- Introduction of **ASP.NET Core**
  - MVC
  - Web API
  - Razor Pages

A Brief History of .Net

27

2017–2019: Transition to Modern .NET

- Consolidation of .NET Core
  - Rapid adoption by the developer community
  - Stable and production-ready versions
- Major Improvements
  - Better performance and reduced memory footprint
  - Simplified project structure (csproj)
  - Built-in dependency injection
- Modern Development Practices
  - Cloud-native and container-friendly
  - Strong support for microservices
  - Improved tooling (CLI, Visual Studio, VS Code)

A Brief History of .Net

28

2020–Today: Unified .NET

- **.NET 5+** unifies:
  - .NET Framework
  - .NET Core
  - Xamarin
- One platform for:
  - Web
  - Cloud
  - Desktop
  - Mobile
  - IoT
- Strong integration with:
  - Cloud
  - Containers
  - Microservices

What is .NET

29

- .NET is a **modern, open-source, cross-platform development platform** for building:
  - Web applications
  - Desktop applications
  - Cloud services
  - APIs
- It provides:
  - A runtime environment
  - A rich class library
  - Multiple programming languages (mainly C#)

Core Components of .NET

30

- **.NET Runtime**
  - Executes compiled code
  - Manages memory (Garbage Collection)
  - Handles exceptions
  - Manages threads
- Two important runtimes:
  - **CLR (Common Language Runtime)** – traditional .NET
  - **CoreCLR** – modern, lightweight, cross-platform

Core Components of .NET

31

- Base Class Library (BCL)
  - Set of reusable classes
  - Collections, I/O,
  - networking,
  - securitySame API across all .NET apps
- Examples:
  - System.Collections
  - System.IO
  - System.Net
  - System.Threading

Languages Supported by .NET

32

- **Main languages**
  - **C#** → primary language for this course
  - Visual Basic .NET
  - F#
- **Key concept: Language interoperability**
  - All languages compile to **Intermediate Language (IL)**
  - IL runs on the .NET runtime
  - Same libraries for all languages
- Result: **language independence**

Languages Supported by .NET

33

- **C# Language**
  - Strongly typed
  - Object-oriented
  - Asynchronous programming support
  - Modern language features (LINQ, async/await)

Types of Applications Built with .NET

34

- **Desktop Applications**
  - Windows Forms
  - WPF
- **Web Applications**
  - ASP.NET Core MVC
  - Web APIs
  - Razor Pages

Types of Applications Built with .NET

35

- **Mobile Applications**
  - .NET MAUI
  - Xamarin
- **Cloud & Services**
  - Microservices
  - REST APIs
  - Cloud-native apps

Cross-Platform Development

36

- .NET applications can run on:
  - Windows
  - Linux
  - macOS
- This makes .NET suitable for:
  - Enterprise systems
  - Cloud environments
  - Containerized deployments

.NET SDK, Runtime & CLI

37

- **.NET SDK**
  - Compilers
  - CLI tools
  - Libraries
  - Templates
- **.NET Runtime**
  - Needed to **run** applications
- **.NET CLI**
  - Command-line interface

Development Tools

38

- **IDEs**
  - Visual Studio
  - Visual Studio Code
- **Package Management**
  - NuGet packages
  - Dependency versioning
- **Build & Deployment**
  - MSBuild
  - CI/CD pipelines
  - Docker support

39

ASP.NET Core Overview

What is ASP.NET Core?

40

- **ASP.NET Core** is a **modern, open-source, cross-platform web framework** developed by **Microsoft** for building:
  - Web applications
  - RESTful APIs
  - Cloud-ready services
- It is part of the **.NET ecosystem** and is designed for:
  - High performance
  - Scalability
  - Maintainability
- ASP.NET Core is the **successor of classic ASP.NET**

What is ASP.NET Core?

41

ASP.NET Core was created to solve limitations of old web frameworks.

- **Main motivations**
  - Windows-only limitation → Cross-platform
  - Heavy configuration → Lightweight & modular
  - Poor performance → High-performance pipeline
  - Tight coupling → Dependency Injection by default

Core Characteristics of ASP.NET Core

42

- **Cross-Platform**
  - Same application runs on: Windows, Linux, macOS
  - Ideal for cloud and server environments
- **High Performance**
  - Built on **Kestrel web server**
  - Optimized request pipeline
  - Minimal overhead

Core Characteristics of ASP.NET Core

43

- **Middleware-Based Architecture**
  - HTTP requests go through a **pipeline**
  - Each middleware handles a specific task: Routing, Authentication, Logging, Error handling
- **Built-in Dependency Injection**
  - No need for external DI libraries
  - Promotes
- **Unified Configuration System**
  - JSON files
  - Environment variables
  - Command-line arguments
  - Centralized configuration

Application Models in ASP.NET Core

44

- **MVC (Model–View–Controller)**
  - Server-side rendering
  - Separation of concerns
  - Suitable for:
    - Enterprise applications
    - Dashboards
    - CRUD systems

Application Models in ASP.NET Core

45

- **Web API**
  - RESTful services
  - Returns JSON / XML
  - Used by:
    - Mobile apps
    - SPAs
    - External systems

Application Models in ASP.NET Core

46

- **Razor Pages**
  - Page-focused model
  - Less boilerplate than MVC
  - Good for small to medium apps

Typical Use Cases of ASP.NET Core

47

- ASP.NET Core is commonly used for:
  - Business applications
  - Administration portals
  - E-commerce backends
  - REST APIs
  - Microservices

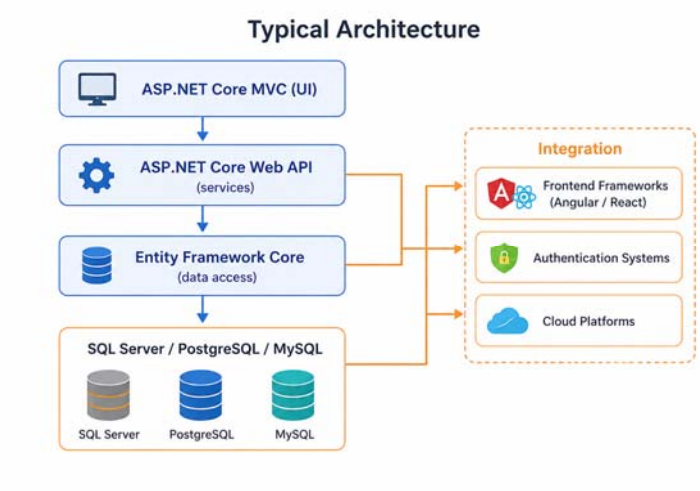
ASP.NET Core in Real Projects

48

- **Typical architecture**
  - ASP.NET Core MVC (UI)
  - ASP.NET Core Web API (services)
  - Entity Framework Core (data access)
  - SQL Server / PostgreSQL / MySQL
- **Integration**
  - Frontend frameworks (Angular / React)
  - Authentication systems
  - Cloud platforms

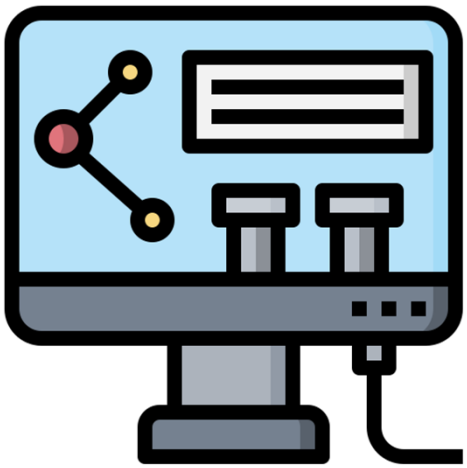
ASP.NET Core in Real Projects

49



Lab 1

50



51

Chapter : ASP.NET Core Architecture

Presentation Outline

52

- Request/response lifecycle
- Middleware and pipeline
- Configuration system
- Dependency Injection (DI)

53

Request/response lifecycle

What is the Request / Response Lifecycle?

54

- The **request / response lifecycle** describes the **complete journey of an HTTP request:**
  - From the client (browser)
  - Through the web server
  - Inside the ASP.NET Core application
  - Back to the client as an HTTP response
- Understanding this lifecycle is **fundamental** for:
  - Debugging
  - Performance optimization
  - Security
  - Middleware configuration

High-Level Flow of a Web Request

55

Step-by-step overview

1. User enters a URL in the browser
2. Browser sends an **HTTP request**
3. Web server receives the request
4. ASP.NET Core processes the request
5. Response is generated
6. Browser receives and renders the response

Actors involved

- **Client:** Web browser
- **Web Server:** Kestrel
- **Application:** ASP.NET Core
- **Optional:** Reverse proxy (Nginx, IIS)

Role of the Web Server

56

- **Kestrel Web Server**

ASP.NET Core applications are hosted by **Kestrel:**

  - Cross-platform web server
  - High performance
  - Handles low-level HTTP details

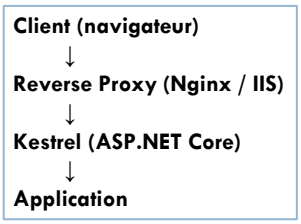
- **Responsibilities**
  - Listen on ports (e.g. 5000 / 5001)
  - Accept incoming connections
  - Forward requests to ASP.NET Core pipeline

Reverse Proxy

57

In production:

- Kestrel is often behind:
  - IIS
  - Nginx
  - Apache
- Roles of reverse proxy:
  - Load balancing
  - SSL termination
  - Security filtering



ASP.NET Core Hosting Model

58

- **Application Startup**
  - The application starts from **Program.cs**
  - The web host is created
  - The HTTP pipeline is configured
- **Key elements in Program.cs**
  - Service registration
  - Middleware configuration
  - Endpoint mapping

Everything begins **before the first request arrives**

Inside the ASP.NET Core Pipeline

59

The Middleware Pipeline Concept

- ASP.NET Core processes requests using a pipeline of middleware components.
- Each middleware:
  - Receives an HttpContext
  - Can:Process the request
  - Pass it to the next middleware
  - Short-circuit the pipeline

Inside the ASP.NET Core Pipeline

60

- **Execution order**
- Middleware executes **in the order it is registered**
- Response flows **back in reverse order**

```
Request →
  Logging →
    Authentication →
      Routing →
        Controller →
      Response →
    Response →
  Response →
Response
```

Endpoint Execution

61

Routing

- ASP.NET Core matches the URL
- Determines:
  - Controller
  - Action
  - Parameters

Controller Action

- Business logic execution
- Interaction with services / database
- Returns:
  - View
  - JSON
  - Status code

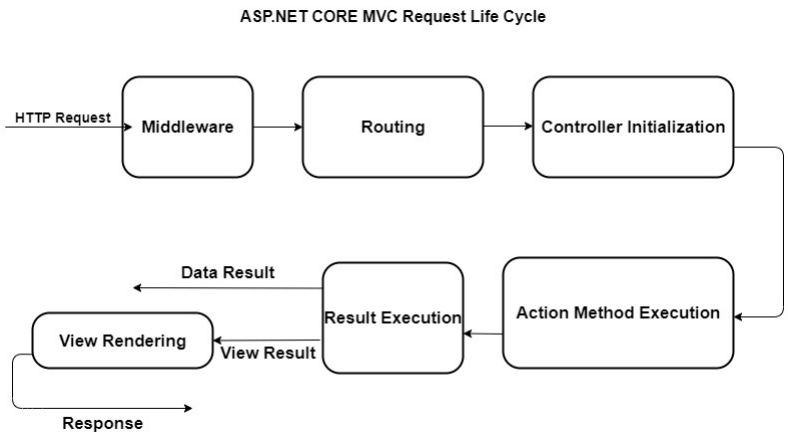
Response Generation

62

- **Response types**
  - HTML (MVC / Razor)
  - JSON (Web API)
  - Static files
  - Redirects
- **Final steps**
  - Response headers are set
  - Status code is returned
  - Response sent to client

Request / Response Lifecycle

63



Request / Response Lifecycle

64

Browser
↓ HTTP Request
Kestrel
↓
Middleware 1
↓
Middleware 2
↓
Routing
↓
Controller Action
↓
View / JSON
↑
Response flows back

65

Middleware and pipeline

What is Middleware?

66

A **middleware** is a **software component** that:

- Is part of the HTTP request pipeline
- Handles requests and/or responses
- Is executed **in sequence**
- Every HTTP request in ASP.NET Core passes through **zero or more middleware components.**

What is Middleware?

67

Key characteristics

A middleware:

- Receives an HttpContext
- Can:
  - Process the request
  - Call the next middleware
  - Stop the pipeline
  - Modify the response

The Middleware Pipeline Concept

68

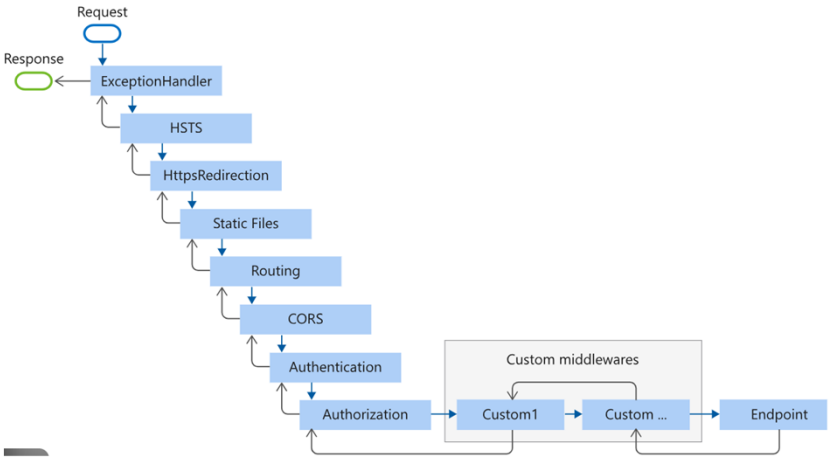
Think of the pipeline as a **chain**:

Request → M1 → M2 → M3 → Endpoint
← M1 ← M2 ← M3 ← Response

- Request flows top → bottom
- Response flows bottom → top
- The **order of middleware registration matters**
  - Security middleware must run **before** controllers
  - Static files must run **before** MVC
  - Error handling must be at the **top**

The Middleware Pipeline Concept

69



Middleware Registration in Program.cs

70

All middleware is registered in **Program.cs** using:

```
var app = builder.Build();
app.UseMiddlewareName();
app.Run();
```

Use vs Run

- Use
  - Passes control to the next middleware
  - Can execute code before and after next()
- Run
  - Terminates the pipelineN
  - No next middleware is executed

Built-in Middlewares in ASP.NET Core

71

- Common built-in middlewares

Ordre	Middleware	Rôle
1	Exception Handling	Gère les erreurs
2	HTTPS Redirection	Force HTTPS
3	Static Files	Sert CSS / JS
4	Routing	Analyse l'URL
5	Authentication	Identifie l'utilisateur
6	Authorization	Vérifie les droits
7	MVC / Endpoints	Exécute l'action

Short-Circuiting the Pipeline

72

What is short-circuiting?

- A middleware may:
  - Stop the request
  - Return a response immediately
  - Prevent next middleware from running
- **Example use cases**
  - Authentication failure
  - Authorization denied
  - Request filtering
  - Custom error responses

Custom Middleware Concept

73

Why create custom middleware?

- Logging
- Request timing
- Header manipulation
- Security checks

```
public async Task InvokeAsync(HttpContext context, RequestDelegate next)
{
    // Before next middleware
    await next(context);
    // After next middleware
}
```

74

Configuration System & Dependency Injection

Why Configuration & DI Matter

75

Problem statement

- In modern applications:
  - Values change between environments
  - Components depend on other components
  - Hard-coded values reduce flexibility
- ASP.NET Core solves this with:
  - A **centralized configuration system**
  - **Dependency Injection (DI)** built-in by default

ASP.NET Core Configuration System

76

- **Configuration** refers to:
  - Connection strings
  - API keys
  - Application settings
  - Environment-specific values
- Example: appsettings.json Structure

```
{
  "AppSettings": {
    "ApplicationName": "MyApp",
    "Version": "1.0"
  },
  "ConnectionStrings": {
    "DefaultConnection": "Server=..."
  }
}
```

ASP.NET Core Configuration System

77

- ASP.NET Core supports **multiple configuration providers.**

Source	Description
appsettings.json	Main configuration file
appsettings.{Environment}.json	Environment-specific
Environment variables	Production-friendly
Command-line arguments	Overrides others
User secrets	Development secrets

Environment Management

78

- ASP.NET Core Environments
  - Development
  - Staging
  - Production
- How environment affects behavior
  - Different config files loaded
  - Different error pages
  - Different logging levels
- Controlled via: ASPNETCORE_ENVIRONMENT

Accessing Configuration in Code

79

Using IConfiguration

- Injected via DI
- Accessible in:
  - Controllers
  - Services
  - Middleware

```
public HomeController(IConfiguration config)
{
    var appName =
config["AppSettings:ApplicationName"];
}
```

Dependency Injection (DI): Core Concept

80

- **Dependency Injection** is a design pattern where:
  - Objects do not create their dependencies
  - Dependencies are provided externally
- Promotes:
  - Loose coupling
  - Testability
  - Maintainability

Dependency Injection (DI): Core Concept

81

- **Without DI (Bad practice)**

```
var service = new EmailService();
```

- **With DI (Good practice)**

```
public HomeController(IEmailService service)
```

Built-in DI Container in ASP.NET Core

82

- **Service Registration:** Services are registered in Program.cs

```
builder.Services.AddTransient<IEmailService, EmailService>();
```

- **Service Lifetimes**

Lifetime	Description
Transient	New instance every time
Scoped	One instance per HTTP request
Singleton	One instance for the app

- **Typical use cases**
  - Transient → lightweight services
  - Scoped → DbContext
  - Singleton → caching, config

Using DI in Application Layers

83

- **DI is used in**
  - Controllers
  - Services
  - Middleware
- Example

```
public class HomeController : Controller
{
    private readonly IEmailService _email;

    public HomeController(IEmailService email)
    {
        _email = email;
    }
}
```

Configuration + DI Together

84

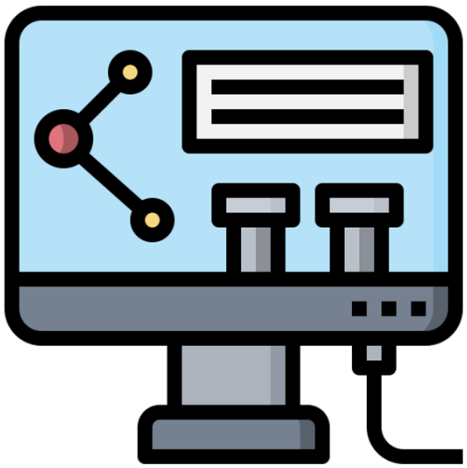
- Binding Configuration to Classes (Strongly typed configuration)

```
builder.Services.Configure<AppSettings>(

builder.Configuration.GetSection("AppSettings"));
```

Lab

85



86

Chapter:
MVC Pattern

Presentation outlines

87

- MVC architecture
- Controllers and actions
- Routing (attribute & conventional)

88

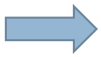
MVC architecture

What is MVC?

89

MVC (Model–View–Controller) is an architectural pattern that separates an application into three distinct responsibilities:

- **Model** → data & business logic
- **View** → user interface (UI)
- **Controller** → request handling & coordination

 MVC is about separation of concerns, not about technology.

Why MVC?

90

- Before MVC:
  - UI logic mixed with business logic
  - Hard to maintain
  - Hard to test
- MVC solves:
  - Code organization
  - Maintainability
  - Team collaboration
  - Testability

High-Level MVC Flow

91

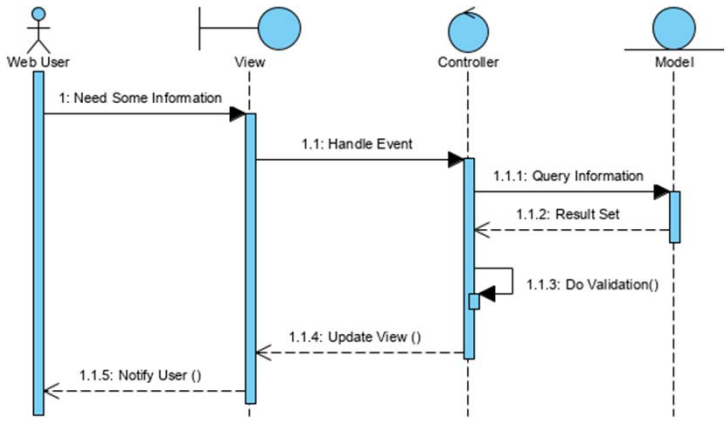
- **Global execution flow**

1. Browser sends HTTP request
2. Controller receives the request
3. Controller interacts with Model
4. Controller selects a View
5. View renders HTML
6. Response sent back to browser

- The **controller is the entry point.**

High-Level MVC Flow

92



Controller

93

- **Role of the Controller**
  - Handles HTTP requests
  - Interprets user actions
  - Calls services / business logic
  - Chooses the response type
- Controller = **traffic controller,** not business logic holder.

Controller

94

Controller responsibilities

- What a controller should do:
  - Routing target
  - Request validation
  - Coordination between Model & View
- What a controller should not do:
  - No UI formatting
  - No database access directly (good practice)

Controller

95

- Example:

```
public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

  - One controller → multiple actions
  - One action → one user interaction
  - Actions return IActionResult

Model

96

In ASP.NET Core MVC, Model can mean different things:

- Domain Model
  - Represents business entities
  - Example: User, Product, Order
- ViewModel
  - Data tailored for the UI
  - Combines multiple models if needed
- Data Model (Entity)
  - Maps to database tables (EF Core)

Model

97

Responsibilities of the Model

What a model should do:

- Holds data
- Applies business rules
- Represents application state

What a model should not do:

- Does not handle HTTP
- Does not know about Views

Model

98

Example:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

View

99

- **Role of the View**
  - Displays data to the user
  - Generates HTML
  - Uses Razor syntax
- View = **presentation only**
- View characteristics:
  - .cshtml files
  - Located in /Views
  - Strongly typed (recommended)

View

100

Responsibilities of the Model

- What a View should do
  - Display data
  - Format UI
- What a View should not do
  - Business logic
  - Database access
  - HTTP handling
- Example:

```
@model Product

<h1>@Model.Name</h1>
```

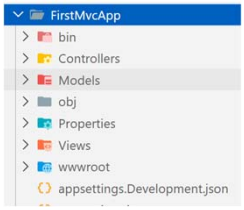
101

Controllers and actions

MVC in ASP.NET Core

102

- Folder structure



- **Convention over configuration**
  - Controller name → View folder
  - Action name → View name
- ASP.NET Core MVC relies heavily on conventions.

Controllers in ASP.NET Core

103

- A controller is:
  - A C# class
  - Located in /Controllers
  - Ends with the suffix Controller

```
public class HomeController : Controller
{
}
```

- Without the suffix Controller, ASP.NET Core will not detect the class.

Actions

104

An **Action** is:

- A **public method** in a controller
- Invoked by an HTTP request
- Returns an HTTP response

```
public IActionResult Index()
{
    return View();
}
```

- Rules for Actions
  - Must be public
  - Cannot be static
  - Usually return IActionResult

IActionResult

105

- IActionResult allows an action to return:
  - HTML
  - JSON
  - Status codes
  - Redirects
- Common Action Results

Method	Response
View()	HTML
NotFound()	404
Json()	JSON
RedirectToAction()	Redirect

Passing Data from Controller to View

106

Technics :

- ViewBag

```
ViewBag.Message = "Hello";
```

- ViewData

```
ViewData["Title"] = "Home";
```

- Model (Recommended)

```
return View(product);
```

⇒ Strongly-typed views are best practice

Model Binding

107

- ASP.NET Core automatically maps:
  - URL parameters
  - Query strings
  - Form data

Action :

```
public IActionResult Details(int id)
```

URL :

```
/Home/Details/5
```

108

Routing

What is Routing?

109

- **Routing** maps an incoming URL to:
  - A Controller
  - An Action
  - Optional parameters
- Without routing, MVC cannot work

Conventional Routing

110

- Default route

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}"
);
```

- How it works
  - URL: /Home/Index
  - Controller: HomeController
  - Action: Index()

Attribute Routing

111

- Routing is defined **directly on controllers or actions**

```
[Route("products")]
public class ProductsController : Controller
{
    [Route("details/{id}")]
    public IActionResult Details(int id)
    {
        return View();
    }
}
```

- URL : /products/details/5
- **When to use Attribute Routing?**
  - REST APIs
  - Explicit URLs
  - Complex routing scenarios

Conventional vs Attribute Routing

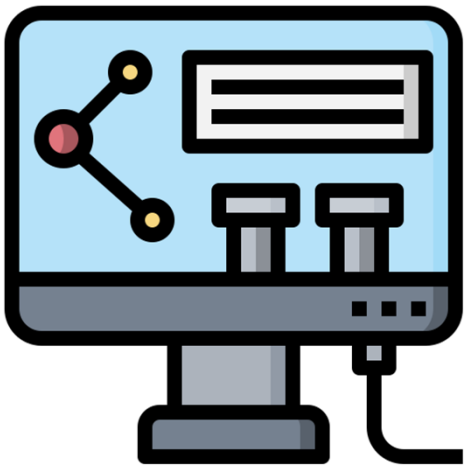
112

Conventional	**Attribute**
Centralized	Explicit
Simple	More control
MVC apps	APIs

- ASP.NET Core supports both together.

Lab

113



114

Chapter :
Razor Views

Presentation outline

115

- Razor syntax
- Layouts, partial views
- Tag Helpers
- Data Annotation
- ViewModels and model binding

What is Razor?

116

Razor is:

- A view engine
- Used to generate dynamic HTML
- Combines C# and HTML in .cshtml files
- Runs on the server, not in the browser.

Basic Razor Syntax

117

- Inline expression

```
<p>@DateTime.Now</p>
```

- Code block

```
@{
   var name = "Ali";
}
<p>@name</p>
```

Basic Razor Syntax

118

- Conditional statements

```
@if (Model.Count > 0)
{
   <p>Students available</p>
}
```

- Loop example

```
@foreach (var student in Model)
{
   <li>@student.Name</li>
}
```

Strongly Typed Views

119

- At the top of the view:

```
@model List<Student>
```

- This enables:
  - IntelliSense
  - Compile-time checking
  - Type safety
- Best practice: Always prefer strongly typed views over ViewBag.

Layouts and Partial Views

120

- **Layouts** define the **common structure** of the application:
  - Header
  - Navigation
  - Footer
  - Scripts
- Default file: Views/Shared/_Layout.cshtml
- RenderBody(): This is where each view's content is injected. @RenderBody()

Layouts and Partial Views

121

- **Partial views:**
  - Reusable UI components
  - Reduce duplication
- Example: Views/Shared/_TeacherCard.cshtml
- Include in a view:

```
<partial name="_TeacherCard" model="teacher" />
```

- Used for:
  - Cards
  - Table rows
  - Repeated UI blocks

Tag Helpers

122

- **Tag Helpers** are server-side components that:
  - Enable server-side logic inside HTML elements
  - Improve readability compared to traditional Razor syntax
  - Use asp- attributes
- Tag Helpers make Razor look more like standard HTML.
- With Tag Helpers:
  - Code More readable
  - Better IntelliSense
  - Closer to pure HTML

Tag Helpers

123

- Tag Helpers Are Enabled in Views/_ViewImports.cshtml

```
@addTagHelper *,
Microsoft.AspNetCore.Mvc.TagHelpers
```

- Server-Side Processing: The browser never sees asp-*

```
<a asp-action="Index">Home</a>
What the browser receives:
<a href="/Home/Index">Home</a>
```

Common Built-in Tag Helpers

124

- Anchor Tag Helper: Used to generate correct links

```
<a asp-controller="Teachers" asp-action="Details" asp-route-id="5">
   View Teacher
</a>
```

Generates : /Teachers/Details/5

- Form Tag Helper: Automatically generates correct action URL

```
<form asp-action="Create" method="post">
```

Forms and Validation

125

- Razor Form Example

```
<form asp-action="Create" method="post">
   <label asp-for="Name"></label>
   <input asp-for="Name" />
   <span asp-validation-for="Name"></span>
   <button type="submit">Save</button>
</form>
```

- Validation Summary

```
<div asp-validation-summary="All"></div>
```

Common Built-in Tag Helpers

126

- Input Tag Helper:
  - Sets name attribute
  - Sets id attribute
  - Applies validation metadata

```
<input asp-for="Name" class="form-control" />
```

- Label Tag Helper

```
<label asp-for="Name"></label>
```

Common Built-in Tag Helpers

127

- Validation Tag Helpers: Display model validation errors

```
<span asp-validation-for="Name"></span>
<div asp-validation-summary="All"></div>
```

- Partial Tag Helper: Renders a partial view

```
<partial name="_TeacherCard" model="teacher" />
```

Data Annotations

128

- **Data Annotations** are attributes applied to model properties that:
  - Define validation rules
  - Define display behavior
  - Influence UI rendering
  - Work with model binding
- They are part of: System.ComponentModel.DataAnnotations

Validation Annotations

129

- Validation annotations define **validation rules.**
- Required
  - Field must not be empty
  - ModelState becomes invalid if empty

```
[Required]
public string Name { get; set; }
```

- StringLength

```
[StringLength(30, MinimumLength = 3)]
public string Name { get; set; }
```

Validation Annotations

130

- Range

```
[Range(18, 65)]
public int Age { get; set; }
```

- EmailAddress

```
[EmailAddress]
public string Email { get; set; }
```

- Compare

```
[Compare("Password")]
public string ConfirmPassword { get; set;
}
```

Display Annotations

131

- Display

```
[Display(Name = "Teacher Full Name")]
public string Name { get; set; }
```

- DisplayFormat

```
[DisplayFormat(DataFormatString = "{0:dd/MM/yyyy}")]
public DateTime DateOfBirth { get; set; }
```

- DataType

```
[DataType(DataType.Password)]
public string Password { get; set; }
```

How Validation Works

132

- User submits form
- Model binding fills the object
- ASP.NET Core validates the object using annotations
- ModelState.IsValid becomes:
  - true → continue
  - false → return view with errors

Displaying Validation Errors in Razor

133

- Validation Summary

```
<div asp-validation-summary="All"></div>
```

- Field-level Validation

```
<span asp-validation-for="Name"></span>
```

ViewModels and Model Binding

134

- A ViewModel:
  - Is tailored specifically for the view
  - Combines multiple models
  - Prevents exposing domain entities directly
- Example:

```
public class TeacherViewModel
{
    public string Name { get; set; }
    public string Subject { get; set; }
    public int StudentCount { get; set; }
}
```

ViewModels and Model Binding

135

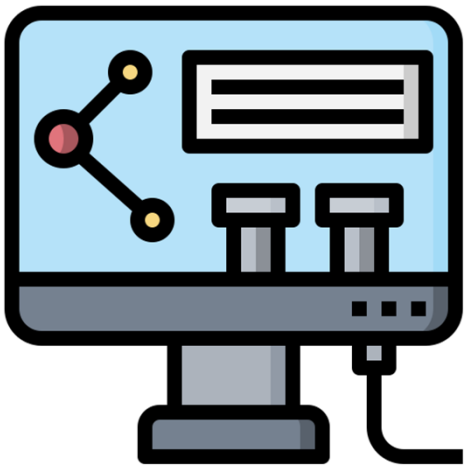
- Model binding:
  - Maps form inputs to C# objects
  - Happens automatically during POST
- Example:

```
[HttpPost]
public IActionResult Create(Teacher teacher)
```

Form fields must match property names.

Lab

136



137

Chapter : Data Access with Entity Framework Core

Presentation outline

138

- ORM principles
- EF Core architecture
- DbContext and entities

What is an ORM?

139

- **ORM = Object-Relational Mapper**
- It is a tool that:
  - Maps database tables : C# classes
  - Converts database rows : C# objects
  - Allows developers to use objects instead of SQL queries
- ORM bridges:

Object-Oriented World ⟺ Relational Database World

What is an ORM?

140

- Without ORM
  - Verbose code
  - Hard to maintain
  - SQL injection risks
  - Tight coupling
- With ORM:
  - Less boilerplate code
  - Strong typing
  - LINQ integration
  - Safer queries
  - Faster development

```
SqlConnection
SqlCommand
SqlDataReader
Manual mapping
```

```
var students = context.Students.ToList();
```

Object vs Relational Mismatch

141

- **Object-Oriented:**
  - Classes
  - Properties
  - Inheritance
  - Navigation properties
- **Relational:**
  - Tables
  - Columns
  - Foreign keys
  - Joins

→ **ORM handles:**

- Mapping relationships
- Translating LINQ to SQL
- Tracking object state

ORM

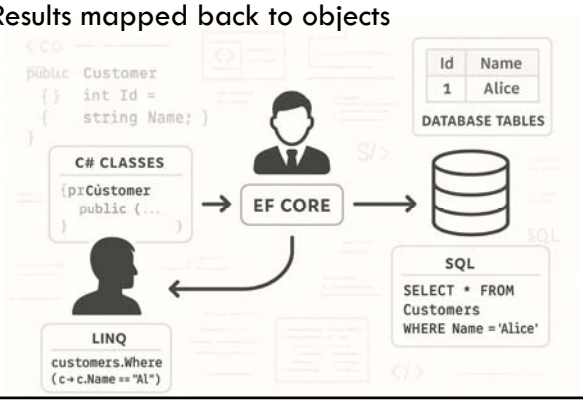
142

- **Advantages of ORM**
  - Productivity
  - Maintainability
  - Strong typing
  - Abstraction from SQL
- **Disadvantages of ORM**
  - Performance overhead
  - Complex queries can be inefficient
  - Requires understanding of generated SQL
- ORM does NOT eliminate the need to understand SQL.

How EF Core Works Internally

143

- Step 1: Developer writes LINQ
- Step 2: EF Core translates LINQ → SQL
- Step 3: SQL sent to database
- Step 4: Results mapped back to objects



What is Entity Framework Core?

144

Entity Framework Core is:

- Microsoft's modern ORM
- Lightweight
- Cross-platform
- Works with:SQL Server, PostgreSQL, SQLite, MySQL,…

EF Core Main Components

145

- Entities
- DbContext
- LINQ Provider
- Change Tracker
- Database Provider

LINQ Provider

146

- The **LINQ Provider** is the component in EF Core that:
  - Interprets LINQ queries written in C#
  - Translates them into SQL
  - Sends SQL to the database
  - Converts results back into C# objects
- It acts as a translator between C# and SQL.

LINQ Provider

147

- Developer Writes LINQ

```
var students = context.Students
                    .Where(s => s.Age > 18)
                    .ToList();
```

- What the LINQ Provider Generates

```
SELECT * FROM Students
WHERE Age > 18;
```

Database Providers

148

- EF Core does not talk directly to the database.
- It uses a provider:
  - Microsoft.EntityFrameworkCore.SqlServer
  - Npgsql.EntityFrameworkCore.PostgreSQL
  - Microsoft.EntityFrameworkCore.Sqlite
- Provider = database-specific translator.

Change Tracking

149

- EF Core tracks:
  - Added entities
  - Modified entities
  - Deleted entities
- When calling:

```
context.SaveChanges();
```

- EF Core:
  - Detects changes
  - Generates appropriate SQL
  - Executes INSERT / UPDATE / DELETE

Entities

150

- An **Entity** is:
  - A C# class
  - Representing a database table
  - Contains properties mapped to columns

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
```

- Mapping Rules (By Convention

C# Class	Database
Class name	Table name
Property	Column
Id	Primary Key

DbContext

151

- DbContext is The main class that coordinates EF Core with the database.
- It represents:
  - A session with the database
  - Unit of work
  - Repository abstraction

```
using Microsoft.EntityFrameworkCore;
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
    public DbSet<Student> Students { get; set; }
}
```

DbContext

152

- Registering DbContext in Program.cs

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer("connection_string_here"));
```

- DbContext Lifetime
  - Usually registered as: `AddDbContext()`
  - Default lifetime = **Scoped.** One DbContext per HTTP request

DbSet

153

- DbSet<T> represents:
  - A table in the database
  - A collection of entities

```
context.Students
```

- Equivalent to:

```
SELECT * FROM Students
```

Basic CRUD with DbContext

154

- Create

```
context.Students.Add(student);
context.SaveChanges();
```

- Read

```
var students = context.Students.ToList();
```

- Update

```
context.Students.Update(student);
context.SaveChanges();
```

- Delete

```
context.Students.Remove(student);
context.SaveChanges();
```

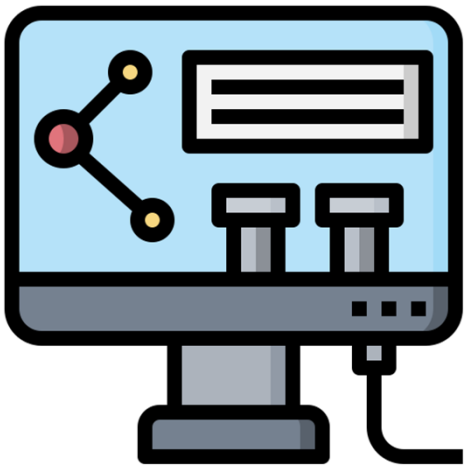
Important concepts

155

- DbContext manages database connection
- DbSet represents tables
- SaveChanges executes SQL
- LINQ is translated to SQL
- EF Core tracks entity state

Lab

156



157

Chapter : Advanced EF Core

Presentation outline

158

- Migrations
- LINQ advanced queries
- Transactions and performance

Migrations in EF Core

159

- Migrations are used to evolve the database schema
- They keep the database structure synchronized with the C# models
- They provide version control for database changes
- Concepts:
  - Model changes → Migration → Database update
  - Code-First approach
  - Migration history table: __EFMigrationsHistory

Migration Workflow

160

- Modify a model (e.g., add a new property)
- Create migration
- Apply migration to database

```
Add-Migration AddCategoryToProduct
Update-Database
```

- EF Core Generate in a Migration
- The Up() Method: Defines the changes to APPLY to the database.

What Does EF Core Generate in a Migration?

161

- The Up() Method: Defines the changes to APPLY to the database.
  - CreateTable()
  - AddColumn()
  - DropColumn()
  - AddForeignKey()
  - CreateIndex()
- The Down() Method: Defines how to REVERT the changes made in Up().
  - DropTable()
  - DropColumn()
  - RemoveForeignKey()
  - DropIndex()

What Does EF Core Generate in a Migration?

162

- **Migration Rollback:** EF Core allows to go back to a previous migration.

```
dotnet ef database update PreviousMigrationName
```

- **Removing the Last Migration:** If migration was created but NOT applied

```
dotnet ef migrations remove
```

- Updating to a Specific Migration: Update database to any migration version:

```
dotnet ef database update MigrationName
```

Migration Scenarios

163

- Common Migration Scenarios
  - Adding a column
  - Removing a column
  - Changing data type
  - Adding relationships (Foreign Keys)
  - Adding indexes
- Migration potential problems:
  - Data loss
  - Migration conflicts
  - Production vs Development migrations
- **Best Practices**
  - Never Edit Production Database Manually
  - Always Review Migration Files Before Applying

Advanced LINQ Queries

164

- Advanced Filtering
  - Multiple Conditions

```
.Where(p => p.Price > 100 && p.Stock > 0)
```

  - Using Contains (Search)

```
.Where(p =>
p.Name.Contains(searchTerm))
```

  - Using Any()

```
.Where(c => c.Products.Any(p => p.Price > 1000))
```

Advanced LINQ Queries

165

- Projection (Select)

```
.Select(p => new
{
    p.Name,
    p.Price
})
```

- Grouping

```
.GroupBy(p => p.CategoryId)
.Select(g => new
{
    CategoryId = g.Key,
    Count = g.Count(),
    AvgPrice = g.Average(p => p.Price)
})
```

Advanced LINQ Queries

166

- Pagination

```
.Skip((page - 1) * pageSize)
.Take(pageSize)
```

- Sorting

```
.OrderBy(p => p.Price)
.OrderByDescending(p => p.CreatedDate)
```

Transactions & Performance

167

- Transactions in EF Core ensures:
  - All operations succeed
  - Or all fail (atomicity)

```
using var transaction =
context.Database.BeginTransaction();
```

Transactions & Performance

168

- **SaveChanges Behavior**
  - SaveChanges() runs inside a transaction automatically
  - Multiple SaveChanges() = multiple transactions
- Best practice:
  - Minimize SaveChanges() calls

```
context.Add(product1);
context.SaveChanges();

context.Add(product2);
context.SaveChanges();
```

```
context.Add(product1);
context.Add(product2);
context.SaveChanges();
```

Transactions & Performance

169

- **Performance Optimization in EF Core:** When working with databases, performance matters.
- Poorly written queries can cause:
  - Slow response times
  - High memory usage
  - Too many database calls
  - Application scalability issues
- Today we focus on **4 key optimization** techniques:
  - AsNoTracking(),
  - Avoid the N+1 Problem,
  - Indexes,
  - Efficient Querying

AsNoTracking()

170

- By default, EF Core **tracks** all retrieved entities. Tracking means:
  - EF monitors changes
  - EF stores objects in memory
  - EF prepares them for update
- This consumes memory and CPU.
- Use **AsNoTracking**() when:Y
  - need only to read data
  - NO need to modify the entity
  - Example: Listing products

```
context.Products.AsNoTracking().ToList();
```

Avoid the N+1 Problem

171

- **The N+1 Problem** happens when:
  - One query loads main entities
  - Then EF executes additional queries for each related entity

```
categories =
context.Categories.ToList();
```

Solution1: Use Projection (Select)

```
var categories = context.Categories
.Select(c => new
{
CategoryName = c.Name,
Products = c.Products.Select(p =>
p.Name).ToList()
}).ToList();
```

Solution2: Use Include()

```
categories = context.Categories
.Include(c => c.Products)
.ToList();
```

Indexes

172

- An index is a database structure that makes data search faster.
- Add Index Using Migration: In EF Core allow to configure indexes in the model. After adding it:

```
Add-Migration AddIndexToProductName
Update-Database
```

- EF will generate CreateIndex() in migration.

Efficient Querying

173

- Common Mistake

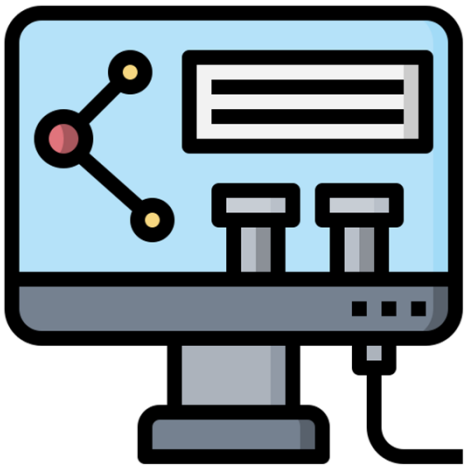
```
context.Products
      .ToList()
      .Where(p => p.Price > 100);
```

- Correct Approach

```
context.Products
      .Where(p => p.Price > 100)
      .ToList();
```

Lab

174



175

Chapter: Web API with ASP.NET Core

Presentation outline

176

- REST architecture
- ASP.NET Core API Controllers
- HTTP status codes
- JSON serialization
- API testing

What is an API?

177

- **API = Application Programming Interface**
- An API allows **applications to communicate with each other.**
- Interactions: Client → Request → Server → Response
- Modern systems rely heavily on APIs.
- Examples:
  - Mobile applications
  - Cloud systems
  - Microservices
  - IoT devices
  - Frontend frameworks (React, Angular)

Web API vs MVC Application

178

MVC Application	Web API
Returns HTML Views	Returns Data
Used for websites	Used for services
Razor Views	JSON responses
Browser rendering	Used by apps/mobile

Example:

MVC:
```
return View(products);
```

API:
```
return Ok(products);
```

What is REST?

179

- REST = Representational State Transfer
- It is the most common architecture for Web APIs.
- REST uses:
  - HTTP methods
  - URLs
  - Stateless communication
  - Standard response formats
- REST APIs typically exchange: **JSON data**

REST Principles

180

- **Stateless**
  - Each request must contain all information.
  - Server **does not store client state.**
  - The server processes the request independently.
  - Example

```
GET /api/products/5
```

- **Resource-based**
  - Everything is treated as a **resource.**
  - Examples

```
/api/products
/api/orders
/api/customers
```

REST Principles

181

- **Standard HTTP Methods**
  - REST APIs use HTTP verbs:

Operation	HTTP Method	Endpoint
Get all products	GET	/api/products
Get product by id	GET	/api/products/5
Create product	POST	/api/products
Update product	PUT	/api/products/5
Delete product	DELETE	/api/products/5

API Response Format

182

- Most APIs return **JSON (JavaScript Object Notation)**
  - Example response:

```
{
  "id": 1,
  "name": "Laptop",
  "price": 1200
}
```

- JSON is:
  - Lightweight
  - Human readable
  - Language independent

Creating an ASP.NET Core Web API

183

- Steps:
  1. Create ASP.NET Core project
  2. Choose **Web API template:** Minimal API (simple) et Controller-based API (structurée).
  3. Add models
  4. Add controllers
  5. Configure routing
- Visual Studio template already includes:
  - Controllers folder
  - API configuration
  - Swagger

```
dotnet new webapi -n MyApiProject --use-controllers
```

API Controller

184

- ASP.NET uses a **Controller** to handle requests.
  - Example: ProductsapiController
  - Controllers usually inherit from: ControllerBase
  - Key attributes: These attributes enable API behaviors:

```
[ApiController]
[Route("api/[controller]")]
```

Routing

185

- Routing defines how URLs map to controller actions.
- In ASP.NET Core Web APIs, routing is commonly defined using **attribute routing**
- Example:
  - GET /api/products: **calls** GetProducts()
  - Example route attribute: [HttpGet]
  - For specific resource: [HttpGet("{id}")]

API Action Example

186

- Example action that returns all products.

```
GET /api/products
```

  - Typical steps:
    1. Query database
    2. Convert to object list
    3. Return result
  - ASP.NET automatically serializes the result to **JSON**.

JSON Serialization

187

- Serialization converts: C# objects → JSON
- Example object:

```
Product product = new Product
{
    Id = 1,
    Name = "Laptop",
    Price = 1200
};
```

- Converted automatically to:

```
{
 "id":1,
 "name":"Laptop",
 "price":1200
}
```

HTTP Status Codes

188

- APIs return **status codes** to indicate result.
- Example:

Code	Meaning
200	OK
201	Created
400	Bad Request
404	Not Found
405	Method Not Allowed
500	Server Error

ASP.NET Helper Methods

189

- ASP.NET Core provides helper methods.
  - Ok()
  - Created()
  - BadRequest()
  - NotFound()
- Example:

```
return NotFound();
```

190

Chapter: Consuming APIs

Presentation outline

191

- Client–server communication
- HttpClient
- Asynchronous programming

What Does "Consuming an API" Mean?

192

- Consuming an API means that an application sends requests to another application or service in order to retrieve or manipulate data.
- Example: A web application may request data such as:
  - Products
  - Orders
  - Weather information
- The data is usually returned in **JSON format.**

Client–Server Communication

193

Client

- Sends requests
- Displays data to the user
- Examples:
  - Web browser
  - Mobile application
  - Desktop application

Client → GET /api/products → Server
Server → JSON Response → Client

Server

- Processes requests
- Accesses database
- Returns responses

HTTP Request Structure

194

- Every HTTP request contains:
  1. **HTTP Method**
  2. **URL**
  3. **Headers**
  4. **Body (optional)**
- Common HTTP methods:

Method	Purpose
GET	Retrieve data
POST	Create new data
PUT	Update data
DELETE	Remove data

JSON Data Exchange

195

- Most Web APIs use **JSON (JavaScript Object Notation)**.
- Reasons:
  - Lightweight
  - Human-readable
  - Supported by most programming languages
- Example:

```
[
  { "id": 1, "name": "Laptop", "price": 1000 },
  { "id": 2, "name": "Mouse", "price": 20 }
]
```

- ASP.NET Core automatically **serializes and deserializes JSON**.

What is HttpClient?

196

- In .NET applications, **HttpClient** is used to send HTTP requests to APIs.
- It allows applications to:
  - Send requests
  - Receive responses
  - Read JSON data
- Example:

```
HttpClient client = new HttpClient();
var response = await
client.GetAsync("https://api.example.com/products");
```

- HttpClient supports: GET, POST, PUT, DELETE

Reading API Responses

197

- After sending a request, we receive an **HTTP response.**
- Example:

```
var response = await client.GetAsync(url);

if (response.IsSuccessStatusCode)
{
    var json = await
response.Content.ReadAsStringAsync();
}
```

- Important response elements:
  - Status code
  - Headers
  - Response body

Deserializing JSON

198

- JSON data must be converted into **C# objects.**
- This process is called **deserialization.**
- Example model:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

- Convert JSON to object:

```
var products =
JsonSerializer.Deserialize<List<Product>>(json);
```

Asynchronous Programming

199

- Web requests can take time to complete.
- If the application waits synchronously, the system may become slow or unresponsive.
- To solve this, .NET uses asynchronous programming.
- Keywords: async, await
- Example:

```
public async Task<IActionResult> GetProducts()
{
    var response = await client.GetAsync(url);
}
```

- Benefits:
  - Better performance
  - Non-blocking operations
  - Improved scalability

Why Asynchronous Calls Matter

200

- Without async: Request → Wait → Response
  - The application is blocked during the wait.
- With async: Request → Continue other work → Response arrives
- Advantages:
  - Faster user experience
  - Better resource utilization
  - Essential for web applications

201

Chapter: Authentication and Authorization

202

- Security fundamentals
- ASP.NET Core Identity
- Role-based authorization

Security fundamentals

203

- Web security is the practice of protecting applications, users, and data from unauthorized access or attacks.
- Modern web applications must protect:
  - user accounts
  - passwords
  - personal data
  - APIs
  - databases
  - sessions and cookies

Security fundamentals
Authentication vs Authorization

204

- **Authentication** answers the question:

"Who are you ?" The system verifies the identity of the user.

Examples: login with username/password, Google login

- **Authorization** answers the question:

"What are you allowed to do?" The system checks permissions after authentication.

Examples: Admin can delete products , Customer can place orders

Security fundamentals
Common Security Threats

205

- **SQL Injection:** Attackers try to execute malicious SQL queries. Example:
- **Cross-Site Scripting (XSS):** Attackers inject JavaScript into pages.
- **Cross-Site Request Forgery (CSRF):** A malicious website tricks a logged-in user into submitting actions.

Security fundamentals
Password Security

206

- Good practices:
  - never store plain passwords
  - always hash passwords
  - use strong password policies
  - use HTTPS
- ASP.NET Core Identity automatically hashes passwords securely.

Security fundamentals
HTTPS and Secure Communication

207

- HTTPS encrypts communication between browser and server.
- Benefits:
  - protects passwords
  - protects cookies
  - prevents data interception
- Production applications should always use HTTPS.

ASP.NET Core Identity

208

- ASP.NET Core Identity is a built-in authentication system in ASP.NET Core. It provides:
  - user registration
  - login/logout
  - password hashing
  - roles
  - authorization
  - account management

Identity Main Components

209

- **IdentityUser:** Represents a user account.

Example properties:

```
public class IdentityUser
{
    public string UserName { get; set; }
    public string Email { get; set; }
}
```

- **IdentityRole:** Represents application roles.

Examples:

- Admin
- Manager
- Customer

Identity Main Components

210

- **UserManager:** Service used to manage users.

Examples:

- create users
- change passwords
- assign roles

- **SignInManager:** Service used for login/logout operations.

Examples:

- password sign in
- sign out
- remember me

Identity Database Tables

211

- Identity automatically creates tables such as:

Table	Purpose
AspNetUsers	stores users
AspNetRoles	stores roles
AspNetUserRoles	relationship user-role
AspNetUserClaims	claims
AspNetUserTokens	tokens

Configuring Identity

212

- Typical configuration in Program.cs:

```
builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();
```

- Authentication middleware checks user identity for every request

```
app.UseAuthentication();
app.UseAuthorization();
```

- Important order:
  1. UseRouting()
  2. UseAuthentication()
  3. UseAuthorization()

Role-Based Authorization

213

- Roles group users by permissions.

Role	Permissions
Admin	manage products/users
Customer	place orders
Manager	manage inventory

- Using the [Authorize] Attribute protect controllers or actions.

```
[Authorize]
public class OrdersController : Controller
{
}
```

Role-Based Authorization

214

- Restricting by Role

```
[Authorize(Roles = "Admin")]
public class AdminController : Controller
{
}
```

- Multiple Roles

```
[Authorize(Roles = "Admin,Manager")]
```

- Allow Anonymous Access

```
[AllowAnonymous]
```

Role-Based Authorization

215

- Policy-Based Authorization provides more advanced authorization rules.

Example

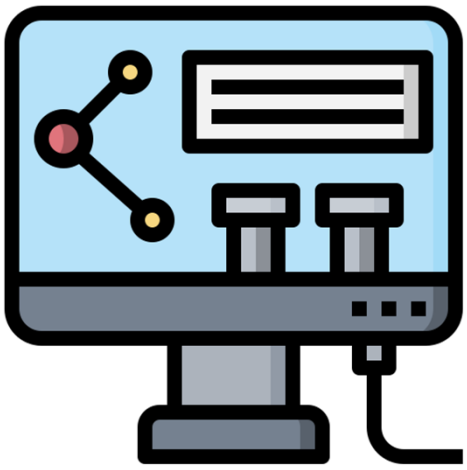
```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireAdminRole",
        policy => policy.RequireRole("Admin"));
});
```

Usage

```
[Authorize(Policy = "RequireAdminRole")]
```

Lab

216



Chapter : Deployment & Configuration

Presentation outline

- Hosting models
- Publishing ASP.NET Core apps
- Environment variables

What is Deployment?

219

- Deployment is the process of making an application available to users after development and testing.
- For an ASP.NET Core application, deployment means:
  - publishing the application files
  - configuring the server
  - preparing databases
  - setting environment variables
  - making the application accessible through a URL

Hosting Models in ASP.NET Core

220

- A hosting model defines how an ASP.NET Core application is hosted and how requests are processed.
- ASP.NET Core applications are usually hosted by:
  - Kestrel
  - IIS
  - Nginx
  - Apache

Kestrel Web Server

221

- Kestrel is the default cross-platform web server included with ASP.NET Core.
- Features:
  - lightweight
  - high performance
  - cross-platform
  - supports HTTP/HTTPS
- Example architecture:
  - Client Browser → Kestrel → ASP.NET Core App

Reverse Proxy Concept

222

- A reverse proxy is a server placed in front of the application server.
- Examples:
  - IIS (Windows)
  - Nginx (Linux)
  - Apache
- Responsibilities:
  - SSL termination
  - load balancing
  - security filtering
  - static file serving
  - request forwarding to Kestrel
- Architecture:
  - Client → IIS/Nginx → Kestrel → ASP.NET Core App

IIS Hosting Model

223

- IIS Overview
  - Internet Information Services is Microsoft's web server for Windows.
- Advantages
  - Easy integration with Windows
  - GUI administration
  - SSL management
  - Process management
  - Widely used in enterprises
- ASP.NET Core Module
  - IIS uses the ASP.NET Core Module to forward requests to Kestrel.

Linux Hosting Model

224

Linux Deployment Overview

- ASP.NET Core applications can run on Linux using:
  - Kestrel
  - Nginx
  - Apache
- Common architecture:
  - Browser → Nginx → Kestrel → ASP.NET Core App

Publishing ASP.NET Core Applications

225

- Publishing prepares the application for deployment.
- It generates:
  - compiled DLL files
  - configuration files
  - static assets
  - Dependencies
- Publish Modes
  - Framework-Dependent Deployment (FDD)
  - Self-Contained Deployment (SCD)

Publish Modes

226

Framework-Dependent Deployment (FDD)

- Characteristics
  - Requires .NET Runtime installed on server
  - Smaller deployment size
- Example:
  - The server already contains .NET 8 Runtime.

Publish Modes

227

Self-Contained Deployment (SCD)

- Characteristics
  - Includes the .NET Runtime
  - Larger deployment size
  - Application can run independently
- Useful when:
  - the server does not have .NET installed

Publish Commands

228

- Basic Publish

```
dotnet publish
```

- Publish in Release Mode

```
dotnet publish -c Release
```

- Publish to Specific Folder

```
dotnet publish -c Release -o ./publish
```

Build Configuration

229

Debug Mode

- Used during development
- Characteristics:
  - slower
  - includes debugging information

Release Mode

- Used in production
- Characteristics:
  - optimized performance
  - smaller output
  - better execution speed

Application Configuration

230

- Configuration allows applications to use different settings depending on the environment.
- Examples:
  - database connection string
  - API keys
  - logging level
  - ports
  - email server
- appsettings.json stores application configuration

Environment-Specific Configuration

231

- ASP.NET Core supports multiple environments.
- Common environments:
  - Development
  - Staging
  - Production
- Files:
  - appsettings.Development.json
  - appsettings.Production.json

Environment Variables

232

- Environment variables are system-level variables used to configure applications.
- Example variables:
  - ASPNETCORE_ENVIRONMENT: Determines the current application environment.
  - ConnectionStrings__DefaultConnection
- Environment Variables advantages
  - better security
  - avoid hardcoding secrets
  - flexible deployment
  - easier cloud deployment

Security Best Practices

233

Never Store Sensitive Data in Code

- Avoid:
  - passwords
  - API keys
  - secret tokens
- Use:
  - environment variables
  - secret managers
  - Azure Key Vault

Production Best Practices

234

- Recommendations:
  - use Release mode
  - enable HTTPS
  - use reverse proxy
  - configure logging
  - protect secrets

Lab

235